

Documentação do Projeto

Simulador do Modelo OSI

Disciplina	Comunicação de Dados
Professor	Prof. Vinícius S. Borges
Semestre	7º semestre
Equipe	Guilherme de Oliveira Mattos · RA 082230009 Luigi Guilherme Pereira Silva · RA 082230025 Paulo Henrique de Carvalho Santos · RA 082230006 Pedro Henrique de Holanda Carvalho · RA 082230005 Tayson Moises Costa do Carmo · RA 082230008

Este documento descreve o funcionamento interno do simulador: a separação entre as camadas, os módulos, a estrutura da unidade de dados, os parâmetros e as convenções que determinam os números produzidos.

Conteúdo

1. Visão geral
2. Separação entre as camadas
3. Diagrama de módulos
4. Estrutura da unidade de dados
5. Interface dos módulos
6. Parâmetros configuráveis
7. Funcionamento interno
8. Convenções de simulação
9. Dependências e ambiente

1. Visão geral

O simulador implementa o modelo OSI de sete camadas em uma rede com três redes locais (Rede A, B e C), quatro roteadores (R1 a R4) e cinco computadores (H1 a H5). Os computadores possuem as sete camadas; os roteadores possuem apenas as camadas 1, 2 e 3.

Uma mensagem gerada na camada 7 da origem desce a pilha, recebe os cabeçalhos de cada camada, atravessa os roteadores e sobe a pilha do destino. Cada ação de cada camada gera uma linha no registro de eventos. A interface gráfica lê esse registro e mostra, passo a passo, o mapa, as pilhas, a unidade de dados e os endereços.

O programa executa os sete casos do enunciado (C1 a C7), que correspondem aos cenários de validação E1 a E7 da especificação, e também envios definidos pelo usuário. Ao final de cada execução, calcula a eficiência $\eta = \text{octetos de dados} \div \text{octetos transmitidos}$.

Caso	Quadros	Octetos transmitidos	Eficiência
C1 Entrega direta	1	92	45,7%
C2 Entrega indireta	4	368	11,4%
C3 Demultiplexação	8	736	11,4%
C4 Falha de enlace	4	368	11,4%
C5 Destino inalcançável	1	92	0
C6 Erro de transmissão	3	276	0
C7 Mensagem longa	12	968	10,3%

2. Separação entre as camadas

2.1 Interface entre camadas adjacentes

Cada camada é uma classe separada em `simulador/camadas.py`, derivada da classe `Camada`, com dois métodos de sentido oposto:

- `descer(...)`: recebe a unidade de dados da camada superior, acrescenta o próprio cabeçalho e chama `descer` da camada imediatamente inferior;
- `subir(...)`: recebe a unidade da camada inferior, remove o próprio cabeçalho e chama `subir` da camada imediatamente superior.

Cada dispositivo guarda as suas camadas no dicionário `camadas`, indexado pelo número da camada. A camada `N` só chama `camadas[N-1]` na descida e `camadas[N+1]` na subida. Nenhuma camada chama uma camada não adjacente. A única exceção é a camada 1, que entrega os bits à camada 1 do dispositivo vizinho, o que representa a transmissão pelo meio físico.

Camada	Na descida chama	Na subida chama
7 Aplicação	<code>camadas[6].descer</code>	não chama ninguém (entrega ao processo)
6 Apresentação	<code>camadas[5].descer</code>	<code>camadas[7].subir</code>
5 Sessão	<code>camadas[4].descer</code>	<code>camadas[6].subir</code>
4 Transporte	<code>camadas[3].descer</code>	<code>camadas[5].subir</code>
3 Rede	<code>camadas[2].descer</code>	<code>camadas[4].subir</code> (computador) ou a própria descida (roteador)
2 Enlace	<code>camadas[1].descer</code>	<code>camadas[3].subir</code>
1 Física	<code>camadas[1].subir</code> do vizinho	<code>camadas[2].subir</code>

2.2 Por que o roteador não consegue ler uma porta

- A classe `Roteador` (`simulador/dispositivos.py`) cria apenas as camadas 1, 2 e 3. Não existe `camadas[4]` em um roteador, então nenhuma camada do roteador pode entregar o pacote a uma camada de transporte.
- As portas são atributos do objeto `Segmento`, que fica guardado dentro do pacote (`pacote.conteudo`). A camada 3 do roteador lê apenas `pacote.ip_destino` e a própria tabela de rotas. Ela nunca abre `pacote.conteudo`.
- Quando o pacote chega à camada 3 de um roteador, o método `subir` chama `descer` da mesma camada: o pacote volta para a camada 2 e segue para o próximo salto, sem subir.
- O conteúdo da mensagem sai cifrado da camada 6 da origem. Mesmo os dados transportados só são legíveis na camada 6 do destino.

2.3 Escopo de cada camada

Camada	O que faz	Unidade
7 Aplicação	Gera a mensagem e identifica os processos pelo nome (navegador, servidorWeb). No destino, entrega a mensagem ao processo.	Mensagem

6 Apresentação	Converte o texto em octetos UTF-8 e cifra o conteúdo. Só a camada 6 do destino decifra.	Mensagem
5 Sessão	Abre a sessão com um identificador (S-0001, S-0002...), mantém e encerra a sessão no destino.	Mensagem
4 Transporte	Numera as portas, segmenta a mensagem e remonta os segmentos em ordem no destino.	Segmento
3 Rede	Insere o par de endereços lógicos e consulta a tabela de encaminhamento.	Pacote
2 Enlace	Insere o par de endereços físicos do salto, numera o quadro e calcula a verificação de erro.	Quadro
1 Física	Converte o quadro em bits e transporta até o vizinho.	Bits

3. Diagrama de módulos

A Figura 1 mostra os módulos e a dependência entre eles. A Figura 2 mostra o fluxo de dados do caso central (C2), da geração da mensagem até a entrega ao processo de destino.

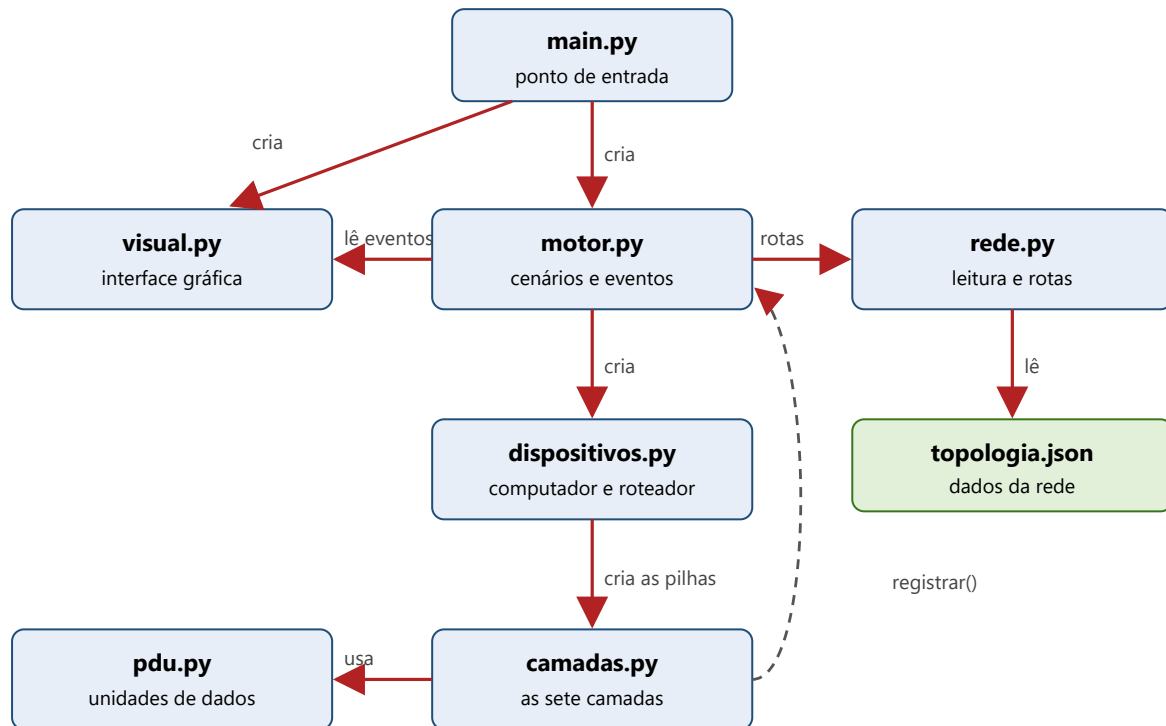


Figura 1 – Módulos do simulador. A linha tracejada indica que as camadas enviam eventos ao motor.

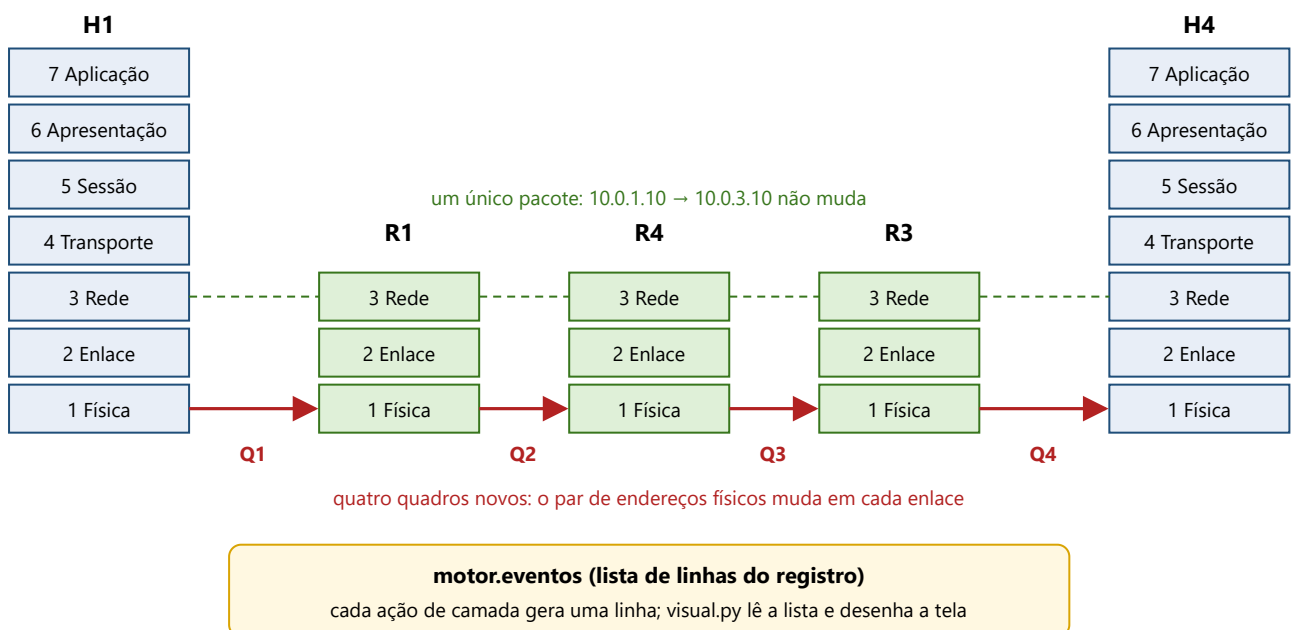


Figura 2 – Fluxo de dados do caso C2: a mensagem desce em H1, sobe até a camada 3 nos roteadores e sobe até a camada 7 em H4.

4. Estrutura da unidade de dados

As unidades de dados ficam em [simulador/pdu.py](#). Todas derivam de **PDU**. Cada unidade guarda a unidade da camada superior em **conteudo**, o que forma o encapsulamento: o quadro contém o pacote, que contém o segmento, que contém a mensagem.

Classe	Campo	Tipo	Função
PDU	conteudo	PDU, bytes ou texto	Conteúdo transportado (a unidade da camada superior ou os dados).
PDU	tamanho	inteiro	Tamanho em octetos: tamanho do conteúdo + tamanho do cabeçalho da camada.
Mensagem	sessao	inteiro	Identificador da sessão. O conteúdo começa com 4 octetos com esse número (por exemplo, "0001").
Segmento	porta_origem, porta_destino	inteiro	Par de portas de origem e destino (por exemplo, 5210 e 443).
Segmento	seq, total	inteiro	Número do segmento e quantidade total de segmentos da mensagem.
Pacote	ip_origem, ip_destino	texto	Par de endereços lógicos, fixo da origem ao destino.
Quadro	mac_origem, mac_destino	texto	Par de endereços físicos do salto corrente.
Quadro	id_quadro	inteiro	Número do quadro (Q1, Q2...) na ordem de transmissão.
Quadro	fcs	inteiro	Verificação de erro (CRC32) calculada na criação do quadro.
Quadro	bits	texto de 0 e 1	Sequência de bits preenchida pela camada 1 na transmissão.

Unidade	Camada	Cabeçalho	Tamanho no caso C2
Mensagem (texto)	7 e 6	0 octetos	42 B
Mensagem com sessão	5	4 octetos	46 B
Segmento	4	8 octetos	54 B
Pacote	3	20 octetos	74 B
Quadro	2	14 de cabeçalho + 4 de finalizador	92 B
Bits	1	—	736 bits

5. Interface dos módulos

5.1 motor.py – classe MotorSimulacao

Método	Devolve	O que faz
carregar_dados(enlaces_caidos)	—	Lê o topologia.json , calcula as rotas e cria os computadores e roteadores.
reiniciar(enlaces_caidos)	—	Zera o registro, os contadores de passos, quadros e sessões e recarrega a topologia.
simular_cenario(nome)	eficiência (%)	Executa o caso C1 a C7 cujo código aparece em nome .
iniciar_transmissao(origem, ip, p_origem, p_destino, texto)	—	Entrega o texto à camada 7 do computador de origem.
finalizar()	eficiência (%)	Calcula a eficiência e acrescenta a linha RESULTADO ao registro.
registrar(dispositivo, camada, ação, descrição, tamanho)	—	Acrescenta uma linha numerada ao registro de eventos.
obter_novo_id_quadro(), nova_sessao()	inteiro	Devolvem o próximo número de quadro e de sessão.
obter_mac_por_ip(ip)	texto ou None	Encontra o endereço físico de um endereço lógico da topologia.
obter_disp_por_mac(mac)	dispositivo	Encontra o dispositivo dono de um endereço físico.
rede_do_ip(ip)	texto ou None	Devolve a rede (por exemplo, 10.0.3.0/24) que contém o endereço.
nome_processo(porta)	texto	Devolve o nome do processo de uma porta (443 → servidorWeb).

5.2 rede.py

Função	Devolve	O que faz
LeitorTopologia.pasta_do_programa()	caminho	Pasta do executável ou, no código-fonte, a pasta do projeto.
LeitorTopologia.carregar(arquivo)	dicionário	Lê o topologia.json da pasta do programa ou, se não existir, a cópia embutida.
calcular_rotas(dados, enlaces_caidos)	dicionário	Calcula a tabela de encaminhamento de cada roteador pelo menor custo.
interface_para(roteador, vizinho)	texto	Devolve a interface do roteador ligada a um vizinho (por exemplo, e1).

5.3 dispositivos.py, camadas.py e pdu.py

Classe	O que faz
Computador(motor, nome, ip, mac, gateway)	Cria as camadas 1 a 7. Guarda o IP, o endereço físico e o gateway.
Roteador(motor, nome, interfaces, rotas)	Cria as camadas 1 a 3. Guarda as interfaces e a tabela de encaminhamento.
CamadaAplicacao ... CamadaFisica	Métodos descer e subir descritos na seção 2.
PDU, Mensagem, Segmento, Pacote, Quadro	Unidades de dados descritas na seção 4.

5.4 visual.py – classe InterfaceSimulador

Monta a janela em Tkinter e reproduz o registro de eventos. Os métodos **simular_cenario** e **simular_manual** pedem ao motor uma execução completa. Em seguida, **executar** lê uma linha de **motor.eventos** por passo e atualiza o mapa (**desenhar_mapa**), as pilhas (**atualizar_pilha**), a unidade de dados (**desenhar_pdu**) e os endereços. A interface não chama métodos das camadas. **salvar_log** grava o registro completo em arquivo.

6. Parâmetros configuráveis

Parâmetro	Onde	Valor padrão	Faixa válida
Cabeçalho da camada 5	<code>pdu.py</code> (Mensagem)	4 octetos	Fixo pela convenção C1.
Cabeçalho da camada 4	<code>pdu.py</code> (Segmento)	8 octetos	Fixo pela convenção C1.
Cabeçalho da camada 3	<code>pdu.py</code> (Pacote)	20 octetos	Fixo pela convenção C1.
Cabeçalho e finalizador da camada 2	<code>pdu.py</code> (Quadro)	14 + 4 octetos	Fixo pela convenção C1.
LIMITE_SEGMENTO	<code>camadas.py</code>	40 octetos	Inteiro maior que zero.
LIMIAR_SEGMENTACAO	<code>camadas.py</code>	64 octetos	Inteiro maior ou igual a zero.
CHAVE_CIFRA	<code>camadas.py</code>	42	Inteiro de 0 a 255.
Arquivo de topologia	<code>rede.py</code>	<code>topologia.json</code> na pasta do executável	Qualquer arquivo com o formato da seção 6.1. Sem o arquivo, vale a cópia embutida.
Velocidade de reprodução	<code>visual.py</code>	Normal (1000 ms por passo)	Lento (2000 ms), Normal (1000 ms) ou Rápido (500 ms).
Mensagens de referência	<code>motor.py</code>	42 octetos (C1 a C6) e 100 octetos (C7)	Texto UTF-8.
Processos	<code>motor.py</code> (PROCESSOS)	5210 e 6120 navegador, 443 servidorWeb	Portas de 1 a 65535.

Os parâmetros dos arquivos `.py` são alterados no código-fonte. O arquivo de topologia é alterado sem mudar o código nem gerar outro executável.

6.1 Formato do `topologia.json`

Seção	Conteúdo
redes	Prefixo de cada rede local e o nome exibido (por exemplo, "10.0.1.0/24": "Rede A").
computadores	Para cada computador: ip, mac, gateway e posicao no mapa.
roteadores	Para cada roteador: posicao e interfaces. Cada interface tem ip, mac e "rede" (rede local ligada) ou "vizinho" (roteador ligado).
enlaces	Enlaces entre roteadores, com os campos de, para e custo.

```
"R1": {"posicao": [170, 120], "interfaces": {
  "e0": {"ip": "10.0.1.1", "mac": "BB:00:00:00:01:00", "rede": "10.0.1.0/24"},
  "e1": {"ip": "10.0.14.1", "mac": "BB:00:00:00:01:01", "vizinho": "R4"},
  "e2": {"ip": "10.0.12.1", "mac": "BB:00:00:00:01:02", "vizinho": "R2"}}
```

7. Funcionamento interno

7.1 Descida na origem

O motor chama **descer** da camada 7 do computador de origem. Cada camada registra a sua ação e passa a unidade para baixo. As linhas abaixo são o início do caso C2:

001		H1		L7		GERA		processo navegador, destino servidorWeb	42 B
002		H1		L6		CODIFICA		octetos UTF-8, conteúdo cifrado	42 B
003		H1		L5		ABRE		sessão S-0001 estabelecida	46 B
004		H1		L4		SEGMENTA		porta 5210 → 443, segmento 1 de 1	54 B
005		H1		L3		ENCAPSULA		10.0.1.10 → 10.0.3.10	74 B
006		H1		L3		ROTEIA		próximo salto 10.0.1.1 pela interface eth0	74 B
007		H1		L2		ENQUADRA		AA::...:01:0A → BB::...:01:00, quadro Q1	92 B
008		H1		L1		TRANSMITE		736 bits no enlace H1-R1	92 B

Na camada 3 do computador, o endereço de destino é comparado com a rede do próprio computador. Se estiverem na mesma rede, a entrega é direta, e o próximo salto é o próprio destino. Caso contrário, o próximo salto é o gateway (10.0.1.1 para a Rede A). A camada 2 obtém o endereço físico do próximo salto e cria o quadro.

7.2 Cada salto no roteador

1. A camada 1 do roteador recebe os bits e registra RECEBE.
2. A camada 2 confere a verificação de erro, retira o pacote e descarta o quadro recebido (DESENQUADRA).
3. A camada 3 procura a rede do destino na tabela de encaminhamento (ROTEIA). A tabela indica o próximo roteador, o endereço lógico dele, a interface de saída e o custo. Se a rede for local ao roteador, a entrega é direta ao computador de destino.
4. A camada 2 cria um quadro **novos**, com outro número e com o endereço físico da interface de saída como origem (ENQUADRA). O pacote passa sem alteração: o par lógico continua o mesmo.

009		R1		L1		RECEBE		736 bits do enlace H1-R1	92 B
010		R1		L2		DESENQUADRA		verificação de erro correta, quadro Q1 descartado	74 B
011		R1		L3		ROTEIA		10.0.3.0/24 via R4, custo 2, interface e1	74 B
012		R1		L2		ENQUADRA		BB::...:01:01 → BB::...:04:00, quadro Q2	92 B

7.3 Tabela de encaminhamento

A função **calcular_rotas** aplica o algoritmo de Dijkstra sobre os enlaces entre roteadores. Para cada roteador e cada rede local, a rota guarda o próximo roteador, o endereço lógico dele, a interface de saída e o custo total. As tabelas resultantes são as da Tabela 3 do enunciado:

	Rede A (10.0.1.0/24)	Rede B (10.0.2.0/24)	Rede C (10.0.3.0/24)
R1	direta, e0	via R2, e2, custo 2	via R4, e1, custo 2
R2	via R1, e0, custo 2	direta, e2	via R3, e1, custo 1
R3	via R4, e0, custo 2	via R2, e1, custo 1	direta, e2
R4	via R1, e0, custo 1	via R3, e1, custo 2	via R3, e1, custo 1

No caso C4, o enlace R1-R4 é retirado do cálculo, e a rota de R1 para a Rede C passa a ser "via R2, e2, custo 3". Se o destino não pertence a nenhuma rede da tabela, a camada 3 registra DESCARTA e o pacote para ali.

7.4 Chegada e remontagem no destino

No computador de destino, a camada 3 confere se o endereço lógico é o dele e entrega o segmento à camada 4. Se a mensagem tem um único segmento, a camada 4 entrega os dados à sessão (DEMULTIPLEXA). Se tem vários, a camada 4 guarda cada segmento em um buffer separado por par de portas (RECEBE) e, quando todos chegam, ordena pelo número do segmento e junta os dados (REMONTA). Só então chama a camada 5. A camada 5 lê o identificador de sessão, a camada 6 decifra o conteúdo e a camada 7 entrega a mensagem ao processo (ENTREGA).

7.5 Quadro corrompido

No caso C6, o motor marca o enlace R4-R3 como enlace com erro. Ao transmitir nesse enlace, a camada 1 inverte o bit do meio da sequência de bits do quadro. A camada 2 do receptor reconstrói os octetos a partir dos bits, recalcula o CRC32 e compara com o valor gravado no quadro. Como os valores são diferentes, registra DESCARTA e encerra o processamento: a camada 3 do R3 não é chamada, não há notificação às camadas superiores e não há retransmissão.

019		R3		L1		RECEBE		736 bits do enlace R4-R3	92 B
020		R3		L2		DESCARTA		verificação de erro falhou, quadro Q3 descartado	92 B

8. Convenções de simulação

As decisões abaixo determinam os números produzidos pelo simulador.

Convenção	Como o simulador trata
Tamanho dos cabeçalhos	Camada 5: 4 octetos. Camada 4: 8. Camada 3: 20. Camada 2: 14 de cabeçalho + 4 de finalizador. As camadas 6 e 7 não acrescentam octetos.
Cifragem	A camada 6 aplica XOR de cada octeto com a chave 42. O número de octetos não muda.
Cabeçalho de sessão na segmentação	A camada 4 recebe os dados da camada 5 já com os 4 octetos de sessão. O cabeçalho de sessão entra uma única vez , no primeiro segmento, e não em cada segmento.
Limite de segmentação	Se os dados recebidos da camada 5 têm mais de 64 octetos, a camada 4 divide em segmentos de no máximo 40 octetos (o último pode ser menor). Até 64 octetos, envia um único segmento. Exemplo: 104 octetos viram 40 + 40 + 24; 46 octetos viram um segmento só. Esse limiar reproduz ao mesmo tempo os valores de E2 (um segmento de 54 B) e de E7 (três segmentos).
Mensagem de referência	C1 a C6 usam o texto <code>GET /index.html HTTP/1.1 Host: servidorWeb</code> (42 octetos). C7 usa um texto de 100 octetos.
Custo do caminho	Soma dos custos dos enlaces entre roteadores. Os enlaces entre computador e roteador não têm custo.
Desempate entre caminhos	O Dijkstra visita os roteadores em ordem de custo e, em caso de empate, em ordem de nome (R1 antes de R2). Um caminho só substitui outro se tiver custo menor. Assim, no empate prevalece o caminho com o menor identificador de roteador.
Numeração dos quadros	Cada salto cria um objeto Quadro novo, numerado Q1, Q2... na ordem de transmissão. A numeração recomeça em cada execução. No caso C3, as duas mensagens usam Q1 a Q8.
Numeração das sessões	Cada mensagem abre uma sessão nova: S-0001, S-0002... A numeração recomeça em cada execução.
Verificação de erro	CRC32 (zlib.crc32) calculado sobre uma representação do cabeçalho do quadro (endereços físicos, número e tamanho do quadro).
Quadro descartado por erro	Descartado pela camada 2 do receptor, sem notificação às camadas superiores e sem retransmissão. Não há entrega.
Octetos transmitidos	Soma dos tamanhos de todos os quadros transmitidos pela camada 1, em todos os enlaces. O número de bits registrado é o tamanho do quadro × 8.
Octetos de dados e eficiência	Octetos de dados são os entregues à camada 7 do destino. $\eta = \text{dados} \div \text{transmitidos}$. Sem entrega, $\eta = 0$.
Numeração do registro	Um passo por ação de camada, a partir de 001, na ordem em que ocorre. As linhas SISTEMA (aviso e resultado) não recebem número e aparecem com "---".
Envio simultâneo (C3)	H1 envia primeiro e H2 em seguida. A camada 4 de H4 separa os fluxos pelo par de portas (5210 → 443 e 6120 → 443).

9. Dependências e ambiente

- **Linguagem:** Python 3. O executável foi gerado com o Python 3.14.7.
- **Bibliotecas:** apenas a biblioteca padrão do Python: `tkinter` (interface), `json` (topologia), `os` e `sys` (caminhos), `ipaddress` (redes e endereços) e `zlib` (CRC32). Nenhuma biblioteca externa é necessária.
- **Sistema operacional:** Windows 10 ou 11.
- **Executável:** gerado com o PyInstaller 6.22.3, em arquivo único, sem janela de console e com o `topologia.json` embutido como cópia de reserva:

```
pyinstaller --onefile --windowed --name SimuladorOSI --add-data "topologia.json;." main.py
```

O executável procura o `topologia.json` primeiro na pasta onde está o `SimuladorOSI.exe`. Só usa a cópia embutida se o arquivo não existir nessa pasta. O registro salvo é gravado na pasta escolhida na janela **Salvar como**, que abre na pasta do executável.

Para executar pelo código-fonte, com o Python 3 instalado, use `python main.py` na pasta do projeto.